Projet – Vehicle Routing Problem with Time Windows (VRPTW)

Guillaume Calderon, Eymeric Dechelette

11/05/2026

Contents

1 Introduction **2**
1.1 Objectif . . . 2
1.2 Choix technologique : pourquoi Rust ? . . . 2

2 Travail réalisé **2**
2.1 Construction d'une solution initiale . . . 2
2.2 Opérateurs de voisinage . . . 3
2.2.1 Relocate . . . 3
2.2.2 Swap . . . 3
2.2.3 2-opt . . . 4
2.2.4 Vérification de validité . . . 4
2.2.5 Remarque sur l'évaluation . . . 4

3 Protocole de comparaison **4**
3.1 Mesures et métriques . . . 4
3.2 Répétitions et paramètres . . . 5
3.3 Machine et environnement . . . 5

4 Résultats **5**
4.1 Comparaisons . . . 5
4.1.1 Graphiques de comparaison . . . 6
4.1.2 Graphiques complémentaires . . . 10

5 Conclusion **11**
5.1 Qualité des solutions . . . 11
5.2 Temps d'exécution . . . 12
5.3 Impact des opérateurs de voisinage . . . 12
5.4 Impact des fenêtres de temps . . . 12
5.5 Limites et perspectives . . . 12

6 Annexe — Comment exécuter **12**
6.1 Avec Cargo . . . 12
6.2 Avec Nix (optionnel) . . . 13
6.3 Comment générer les graphiques . . . 13

1 Introduction

1.1 Objectif

Le but du projet est de produire des solutions valides au Vehicle Routing Problem with Time Windows (VRPTW) et de comparer plusieurs approches de recherche locale / métaheuristiques sur des instances fournies.

La comparaison porte sur :

- Temps d'exécution
- Nombre de véhicules utilisés
- Distance totale parcourue

Méthodes comparées :

- Marche aléatoire
- Descente (hill-climbing)
- Recherche tabou
- Recuit simulé

1.2 Choix technologique : pourquoi Rust ?

Nous avons choisi Rust pour :

- Performance : compilation native, exécution rapide.
- Mesures plus propres : moins d'effet d'un interpréteur ou d'une VM sur les temps.
- Robustesse : gestion d'erreurs explicite, ce qui permet au LLM de développer plus efficacement et mieux cibler leur erreurs.
- Habitude : nous avons déjà une expérience avec Rust, ce qui nous a permis de démarrer rapidement.
- Reproductibilité : environnement optionnel via `flake.nix`.

2 Travail réalisé

2.1 Construction d'une solution initiale

Les méthodes comparées partent d'une solution initiale construite par un procédé simple :

- Ordre initial des clients généré aléatoirement

- Insertion séquentielle dans la tournée courante
- Si insertion impossible (capacité ou temps), création d'une nouvelle tournée

Cette partie sert de base commune afin de comparer les méthodes de manière équitable.

2.2 Opérateurs de voisinage

Les méthodes utilisent des mouvements de voisinage pour produire des solutions voisines. Nous comparons trois configurations de voisinage :

- `relocate`
- `relocate` + `swap`
- `relocate` + `swap` + `2-opt`

Dans tous les cas, un mouvement est accepté uniquement si la solution obtenue reste valide (capacité et fenêtres de temps).

2.2.1 Relocate

Principe : déplacer un client d'une position vers une autre.

Deux variantes sont utilisées :

- relocate intra-tournée : retirer un client d'une tournée et le réinsérer à une autre position dans la même tournée
- relocate inter-tournées : retirer un client d'une tournée et l'insérer dans une autre tournée

Ce mouvement permet d'ajuster finement l'affectation des clients aux véhicules et l'ordre de visite.

2.2.2 Swap

Principe : échanger deux clients.

Deux variantes sont utilisées :

- swap intra-tournée : échanger la position de deux clients dans une même tournée
- swap inter-tournées : échanger un client d'une tournée A avec un client d'une tournée B

Ce mouvement est utile pour corriger des "mauvaises associations" (un client dans la mauvaise tournée) ou pour réduire la distance localement.

2.2.3 2-opt

Principe : choisir deux arêtes dans une tournée et inverser le segment intermédiaire afin de supprimer des croisements et raccourcir le trajet.

Dans ce projet, `2-opt` est appliqué en intra-tournée uniquement.

2.2.4 Vérification de validité

Après chaque mouvement, on vérifie :

- capacité : la somme des demandes de chaque tournée ne dépasse pas la capacité du véhicule
- fenêtres de temps : pour chaque tournée, on simule la progression temporelle dans l'ordre de visite
  - le temps augmente avec le temps de trajet
  - si arrivée avant `ready_time`, attente jusqu'à `ready_time`
  - si arrivée après `due_time`, la tournée est invalide
  - le temps de service est ajouté à chaque client

Si la solution est invalide, le voisin est rejeté.

2.2.5 Remarque sur l'évaluation

Pour comparer équitablement les voisinages, on utilise la même fonction d'évaluation et la même règle de comparaison des solutions.

3 Protocole de comparaison

3.1 Mesures et métriques

Pour chaque instance et chaque configuration, on mesure :

- Qualité
  - K (nombre de véhicules)
  - D (distance totale)
  - Faisabilité (valide / invalide)
- Performance

  - Temps d'exécution (ms)

- Statistiques d'exploration
  - `Solutions` (nombre de solutions générées)
  - `NeighborsAttempted` (nombre de voisins tentés)
  - `NeighborsAccepted` (nombre de voisins acceptés)

3.2 Répétitions et paramètres

- Répétitions : 1 exécution par configuration (dataset, TW, méthode, voisinage)
- Seed : non fixé, générateur aléatoire initialisé par le système

Paramètres utilisés :

- Budgets par méthode
  - Random Walk : `iterations=5000`
  - Descente : `iterations=5000`
  - Recherche tabou : `iterations=500`, `tabu_tenure=20`, `attempts_per_iter=50`
  - Simulated annealing : `iterations=10000`, `initial_temp=500.0`, `cooling_rate=0.995`

3.3 Machine et environnement

- CPU, RAM, OS : AMD Ryzen 5 5600H 12 coeur 4.2Ghz/32G DDR4/Linux (NixOS)
- Version Rust : 1.94.1

4 Résultats

Les résultats complets (toutes instances) sont fournis par le programme et sauvegardés dans `outputs/<instance>/results.csv`.

4.1 Comparaisons

Le programme affiche :

- un tableau complet par instance (avec K, D, temps d'exécution, voisinages et paramètres)
- deux tableaux de synthèse par instance et séparés avec et sans fenêtres de temps :
  - Qualité : distance moyenne (ordre du meilleur au moins bon)

  - Rapidité : temps moyen (ordre du plus rapide au plus lent)

Chaque ligne correspond à une configuration :

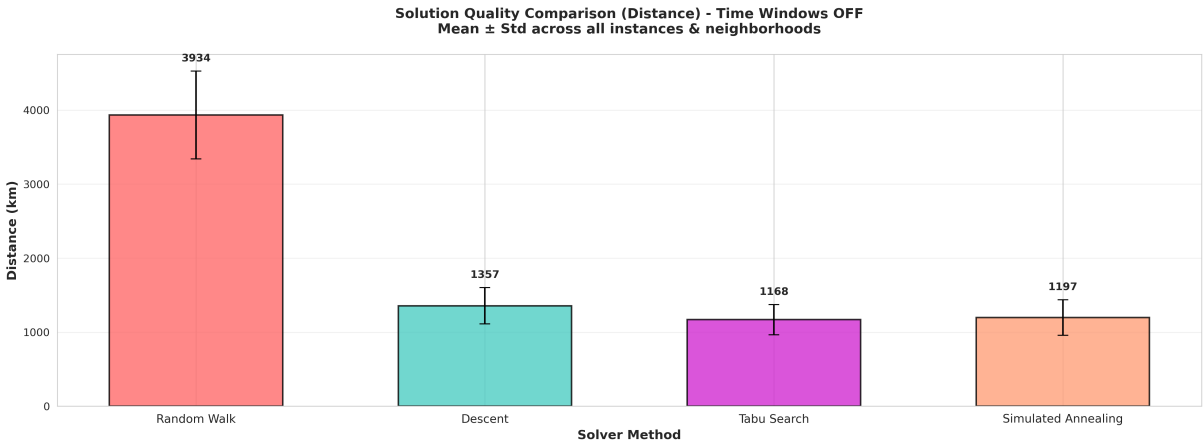
- Méthode (Random Walk / Descente / Tabou / Recuit)
- Voisinage utilisé (relocate, relocate+swap, relocate+swap+2-opt)
- Paramètres (itérations, température, tenure, etc.)

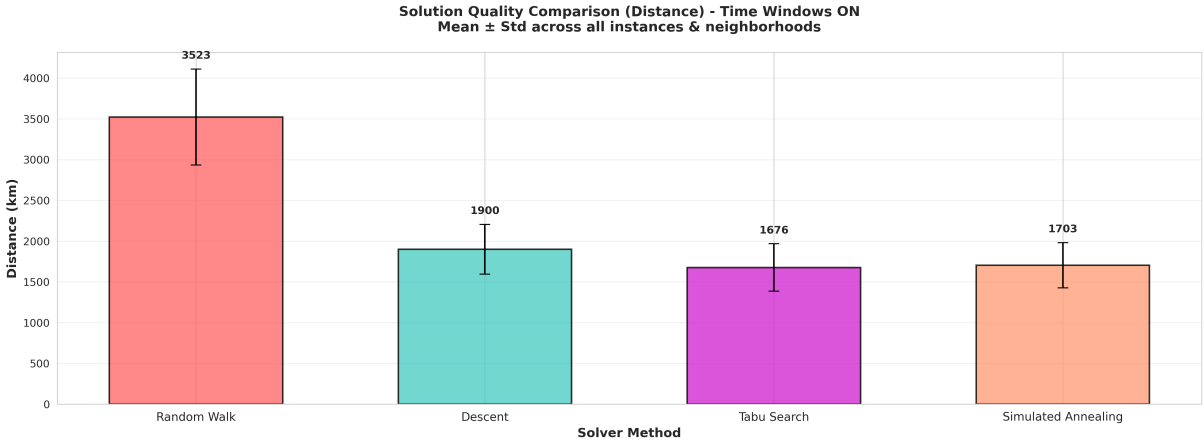
La colonne `DeltaBest` indique l'écart par rapport au meilleur résultat du tableau (plus faible = meilleur).

Les tableaux sont générés directement dans la sortie CLI, et les valeurs détaillées sont sauvegardées dans :

- `outputs/<instance>/results.csv`

4.1.1 Graphiques de comparaison





Observations clés :

- TW OFF : Tabu Search et Simulated Annealing surpassent Descent de 13-15%
- TW ON : Écart réduit, mais Tabu Search et SA restent meilleurs
- Random Walk : Clairement la pire approche
- Conclusion : Les métaheuristiques sont indispensables pour ce problème

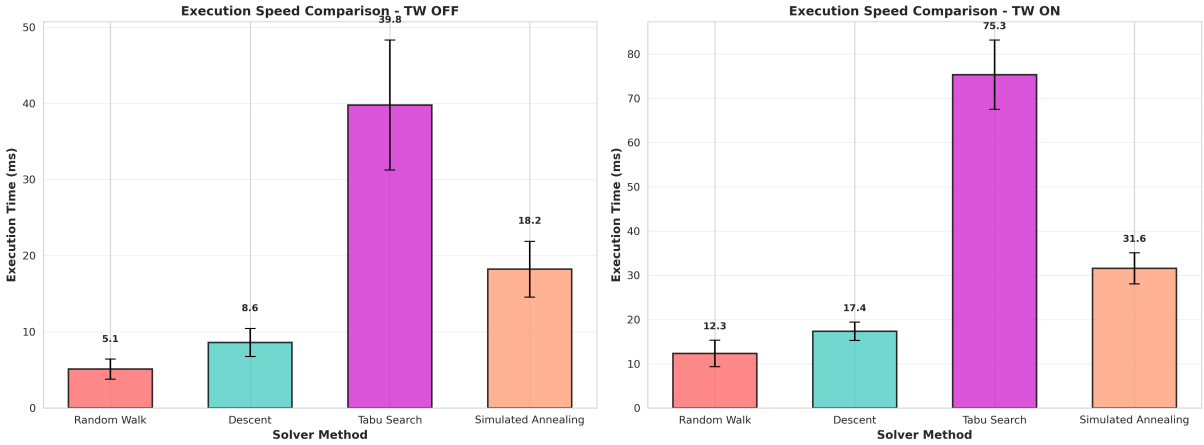
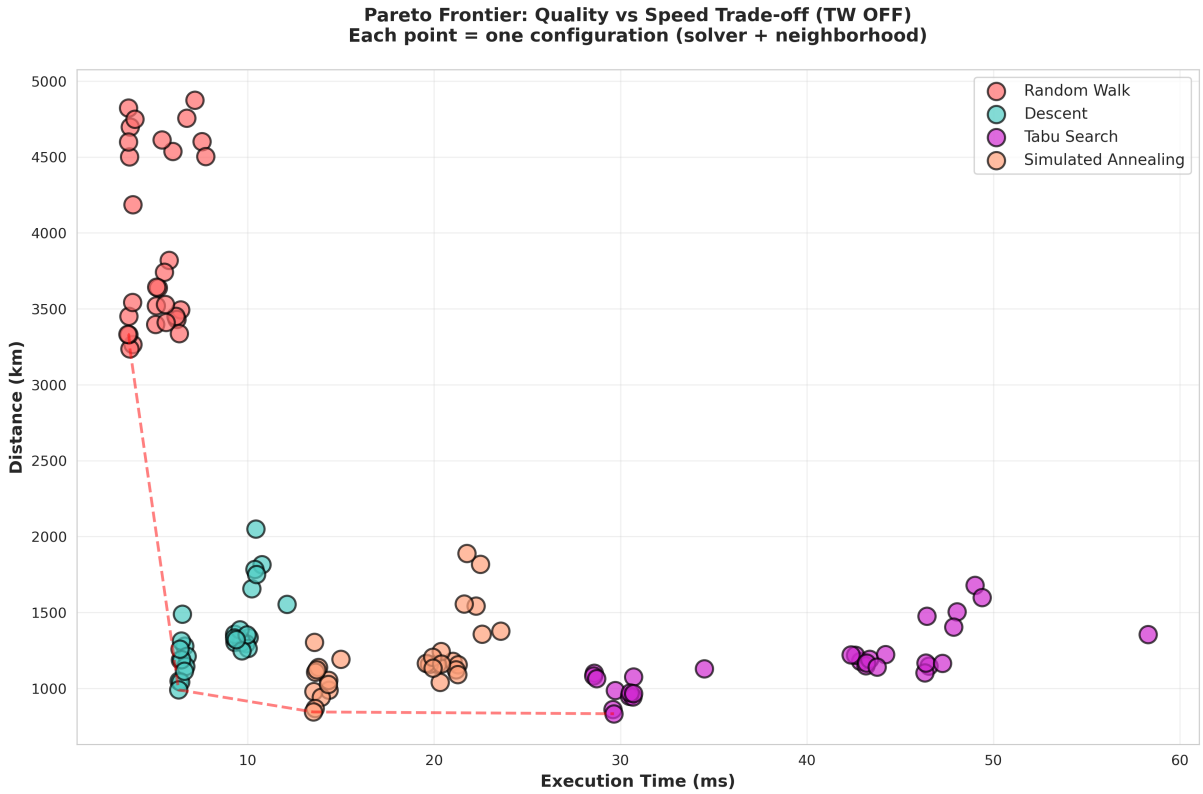
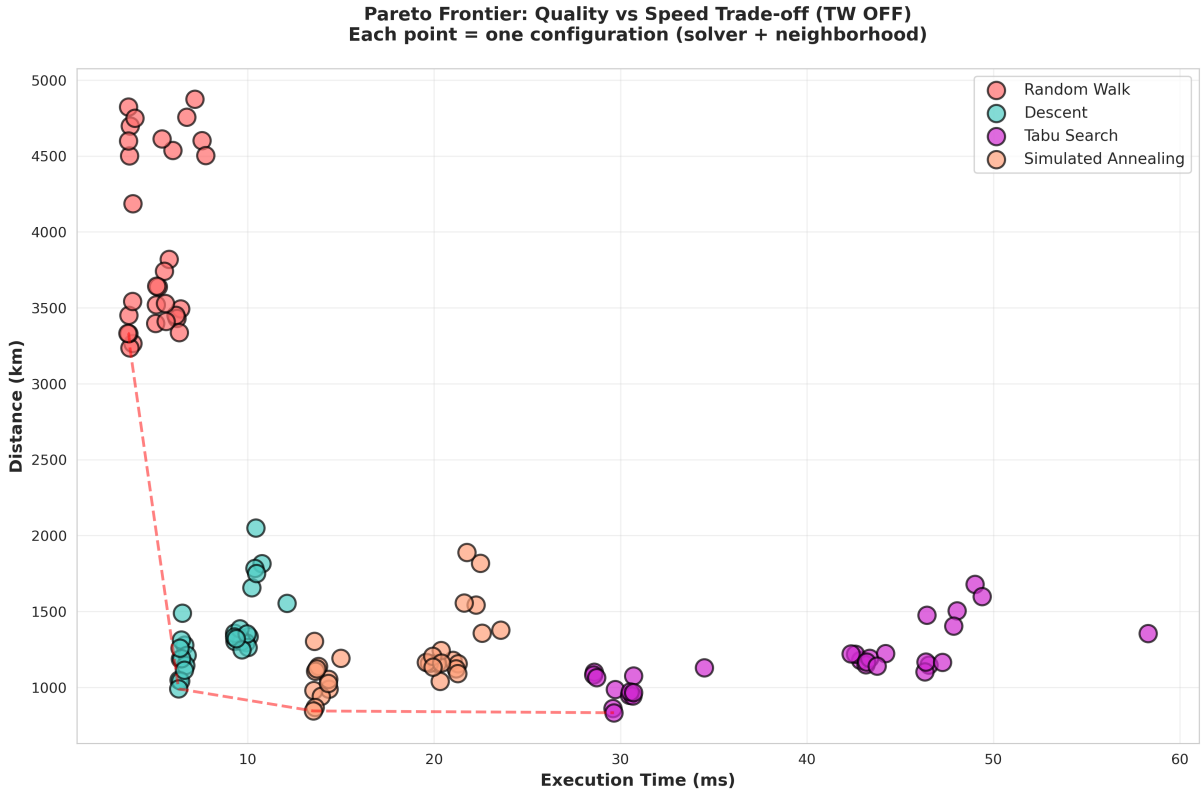


Figure 1: 03_speed_by_solver

Observations :

- Random Walk : Plus rapide mais mauvaise qualité
- Descent : Rapide et meilleur que RW
- Tabu Search : Plus lent mais meilleures solutions
- Simulated Annealing : Bon compromis avec excellente qualité
- Conclusion : Trade-off temps/qualité : Tabu Search 7× plus lent mais 3× meilleur en distance


Pareto Frontier: Quality vs Speed Trade-off (TW OFF)
Each point = one configuration (solver + neighborhood)
Random Walk
Descent
Tabu Search
Simulated Annealing
Distance (km)
Execution Time (ms)
5000
4500
4000
3500
3000
2500
2000
1500
1000
10
20
30
40
50
60


Pareto Frontier: Quality vs Speed Trade-off (TW OFF)
Each point = one configuration (solver + neighborhood)
Random Walk
Descent
Tabu Search
Simulated Annealing
Distance (km)
Execution Time (ms)
5000
4500
4000
3500
3000
2500
2000
1500
1000
10
20
30
40
50
60

Interprétation :

- Chaque point représente une configuration (solveur + voisinage)
- Points en bas-gauche = meilleur compromis
- La courbe rouge pointillée = solutions Pareto optimales (impossible d'améliorer sans sacrifier)
- Meilleur choix pratique : Simulated Annealing
- Alternative plus qualitative : Tabu Search

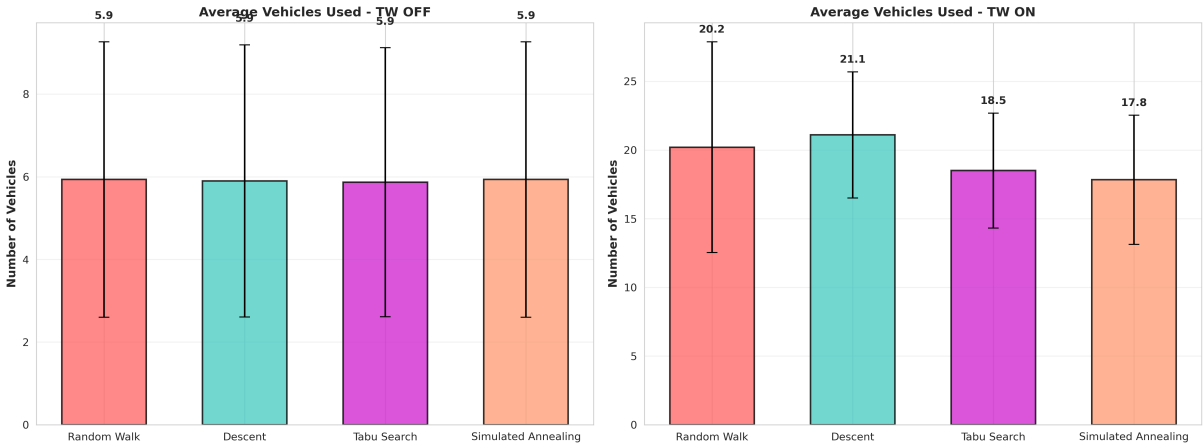
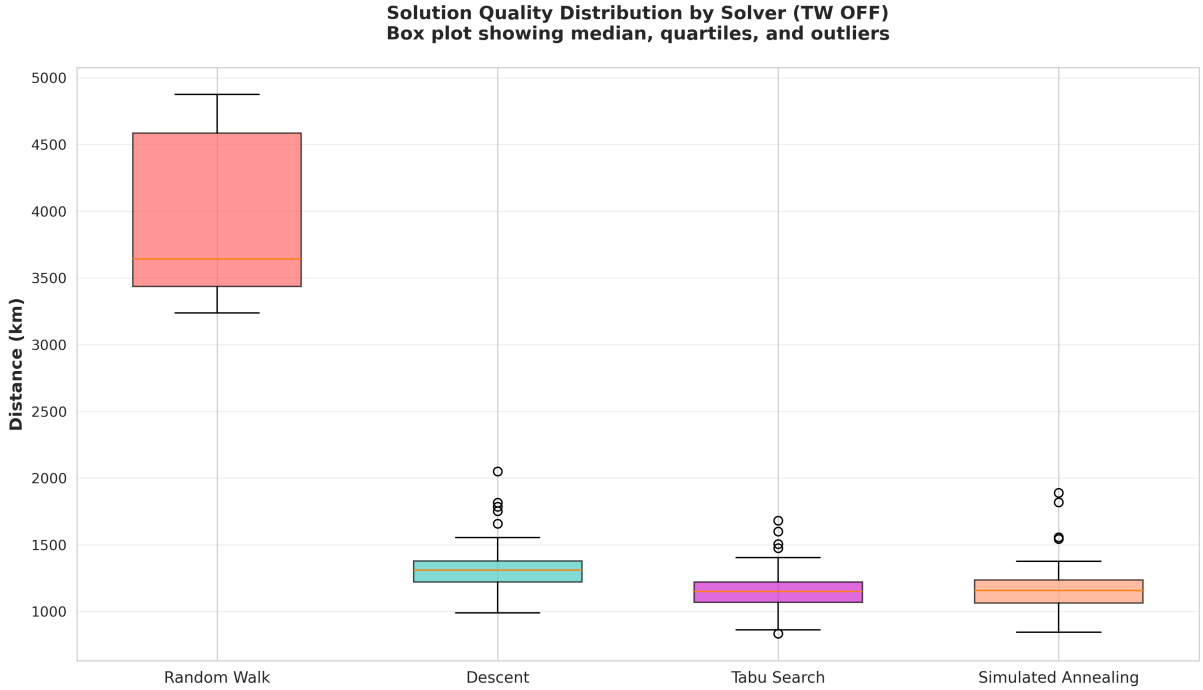


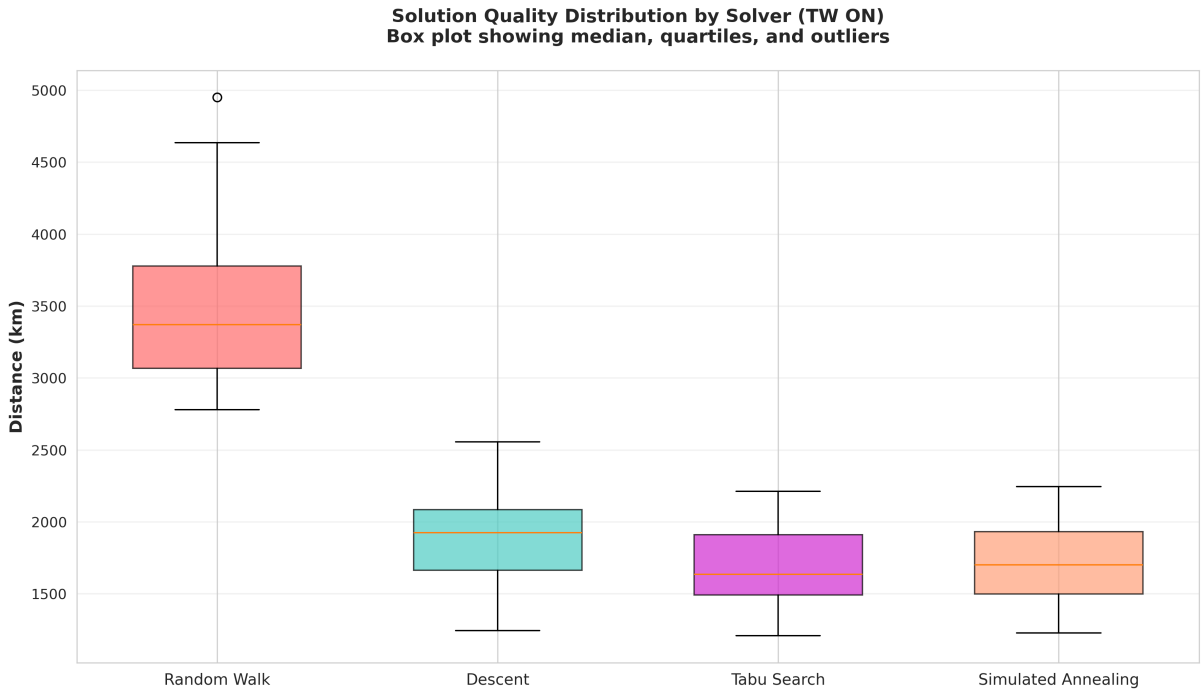
Figure 2: 09_vehicles_used

Observations clés :

- TW OFF : Tous les solveurs utilisent ~8-9 véhicules
- TW ON : Véhicules requís augmente à ~20-30 pour tous
- Augmentation : Facteur 2.5-3× causée par les contraintes temporelles
- Conclusion : TW rend le problème significativement plus difficile (plus de fragments)

4.1.2 Graphiques complémentaires





Observations :

- Random Walk : Boîte très large = très instable
- Descent : Boîte étroite = stable mais moins bon

- Tabu/SA : Boîtes étroites = résultats reproductibles
- Conclusion : Métaheuristiques = plus fiables

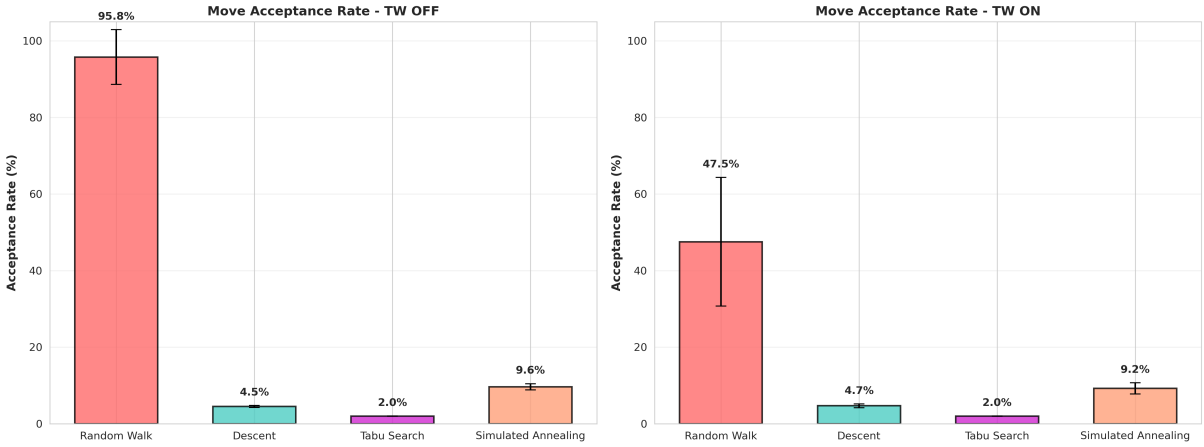


Figure 3: 10_acceptance_rate

Observations :

- Random Walk : ~99% acceptation (très diversifié)
- Descent : ~4% acceptation (très sélectif)
- Tabu Search : ~2% acceptation (cherche activement)
- SA : ~10% acceptation (équilibre diversité/intensité)
- Conclusion : Stratégies d'acceptation très différentes

5 Conclusion

Cette étude comparative a permis d'évaluer quatre approches de recherche locale sur le VRPTW, en faisant varier les opérateurs de voisinage et la présence de fenêtres de temps.

5.1 Qualité des solutions

Les métaheuristiques : recherche tabou et recuit simulé
Produisent systématiquement les meilleures solutions. Sans fenêtres de temps, elles surpassent la descente en distance totale, et restent supérieures même avec les contraintes temporelles actives. La marche aléatoire confirme son rôle de simple baseline : ses résultats sont deux à trois fois moins bons que ceux des métaheuristiques, ce qui illustre l'importance d'une stratégie d'exploration guidée.

5.2 Temps d'exécution

Le coût de cette qualité est réel : la recherche tabou est environ 7 fois plus lente que la descente, qui reste elle-même plus rapide que le recuit simulé. Le recuit simulé offre cependant le meilleur compromis temps/qualité, atteignant des distances comparables à la recherche tabou avec un temps d'exécution nettement inférieur.

5.3 Impact des opérateurs de voisinage

L'ajout du 2-opt apporte une amélioration significative, en particulier pour les instances sans contraintes temporelles. L'opérateur swap contribue de façon plus modérée. Ces résultats confirment l'intérêt de combiner plusieurs types de mouvements pour diversifier l'exploration.

5.4 Impact des fenêtres de temps

L'activation des fenêtres de temps augmente le nombre de véhicules nécessaires d'un facteur 2,5 à 3, rendant le problème structurellement plus difficile. Le taux de voisins valides chute, ce qui réduit l'efficacité des opérateurs et rapproche les performances des différentes méthodes.

5.5 Limites et perspectives

Les résultats reposent sur une seule exécution par configuration, sans graine fixée, ce qui limite leur reproductibilité statistique. Des expériences avec plusieurs répétitions et des seeds fixes permettraient de quantifier la variance et de valider les comparaisons. Par ailleurs, les paramètres (tenure, température initiale, taux de refroidissement) n'ont pas été optimisés systématiquement — un réglage par recherche en grille ou par méthode bayésienne pourrait améliorer sensiblement les résultats.

6 Annexe — Comment exécuter

6.1 Avec Cargo

Depuis la racine du projet (dossier contenant `Cargo.toml`) :

- Exécution (debug) : `cargo run`
- Exécution (optimisée) : `cargo run --release`

Remarque : le programme charge pour l'instant une instance via un chemin relatif (ex : `data/data101.vrp`). Il faut lancer la commande depuis la racine du dépôt pour que les chemins vers `data/` fonctionnent.

6.2 Avec Nix (optionnel)

- Entrer dans le shell : `nix develop`
- Puis exécuter : `cargo run --release`

Alternative (en une commande) : `nix develop -c cargo run --release`

6.3 Comment générer les graphiques

Dépendances requises :

```
pip install pandas matplotlib numpy seaborn # ou nix develop
```

Génération :

```
python3 generate_plots.py
```

Les graphiques sont générés dans `figures/`.